

Project 1: LLM Serving

Project Proposal - Track B: Customer Support Chatbot

Arya Bhosale	A0314484A
Reiner Anggriawan Jasin	A0314502W
Sarthak Kumar	A0330356L
Thet Su Win	A0304980A

1. Introduction

This proposal describes our approach to **Track B: Customer Support Chatbot** of the CS4262/5462 Machine Learning Systems Project 1. This track requires building a high-throughput LLM serving engine optimised for massive concurrency and strict latency SLAs. The benchmark issues requests at a concurrency of **128** against 40,000 customer service inquiries split into training, validation, and test sets. Performance is evaluated across end-to-end Avg & P50/P95 latency, throughput (requests per second), and average perplexity.

2. Problem Statement & Motivation

Customer support applications require an LLM to handle hundreds of simultaneous multi-turn sessions while keeping Time-To-First-Token (TTFT) low. The provided starter kit exposes the `POST /v1/chat/completions` endpoint via vLLM with default settings, but leaves several inefficiencies unaddressed:

- **Redundant computation from repeated queries.** Common queries such as “What is your return policy?” appear many times, yet each triggers a full prefill from scratch, wasting GPU compute.
- **Suboptimal KV-cache eviction.** The default LRU policy evicts cache blocks without considering session state, potentially discarding the KV-cache of an ongoing session and forcing costly recomputation.
- **High TTFT from large batched prefills.** Processing each prefill as a single chunk blocks decode steps for all other active sessions, inflating TTFT under high concurrency.
- **Missed speculative decoding opportunity.** Customer support responses are typically short and formulaic, making them well-suited for speculative decoding to speed up the decode phase.
- **Cold-start latency for repeated questions.** When a popular question returns after its cache entry has been evicted, the engine pays the full prefill cost again despite having answered it before.

Our goal is to address these inefficiencies to achieve lower TTFT and higher throughput without degrading output quality.

3. Proposed Optimisations

We propose five complementary optimisations, each targeting a distinct inefficiency. Figure 1 shows how they compose into a unified serving stack. Optimisations 1, 2, and 3 operate at different layers of the serving stack and share no implementation dependencies, allowing team members to work on them in parallel. The only exception is Optimisation 5, which depends on the session-state tracking introduced in Optimisation 4. We will therefore develop Optimisation

4 first before starting Optimisation 5.

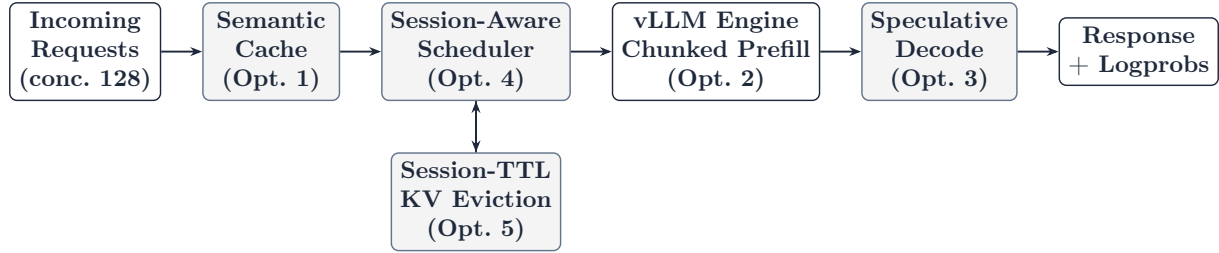


Figure 1: High-level architecture of our Track B serving engine. The five proposed optimisations layer on top of vLLM’s core inference engine. Shaded nodes (grey background) represent new components added to the baseline; white nodes represent the existing vLLM infrastructure. Note that the optimisation numbers reflect the order in which they are described in this proposal, not their position in the pipeline.

3.1 Optimisation 1 — Semantic Response Cache with Exact + Fuzzy Matching

One of the most effective way to optimize customer support traffic is through caching. In customer support system, many users would ask similar or identical questions repeatedly. Questions like "Where is my order?" will appear numerous times. Generating a new response for the same question is inefficient. By reusing previously generated answers through caching system, we aim to reduce the computational cost and improves the response time.

Our initial layer of the cache will expect exact matching, Our system will store previous conversations in an in-memory dictionary that follows an LRU policy. Each entry will be marked by the SHA-256 hash created from the full chat history used as a key for the dictionary. Prior to the hashing step, the chat history including the current message will be normalized by converting it to lowercase and removing unnecessary whitespace so that small formatting differences do not affect the match. With caching, we can trigger a simple lookup from the memory instead of sending request to the LLM.

However, exact matching cannot detect questions that have the same meaning that are expressed differently, like "Where is my package?" and "Can you track my order?" which produces different hashed for lookup. To solve this, our next later of caching will be based on semantic similarity. Our semantic caching will convert each user query into a vector representation using a lightweight bi-encoder which can run on CPU (e.g. `all-MiniLM-L6-v2`). These vectors are stored in a FAISS index for a more efficient similarity search. When a new query arrives, the embedding is compared with previously stored queries using cosine similarity. If the similarity exceeds certain threshold, the system will assume the questions are semantically equivalent and returns the cached response.

To maintain efficiency and prevent the cache from growing indefinitely, our sematic cache applies a time-aware LRU policy combined with a time-to-live (TTL) value, typically about 30 minutes. This ensure that the caching entries will be automatically removed after some period. Additionally, frequently accessed entries receive special protection from automatic removal. If a query is accessed at least five times within five minutes, the entry is temporarily pinned in the cache until traffic drops. This prevents popular queries from being removed and reinserted during periods of high traffics.

Expected impact: Assuming a 30–40% cache hit rate (conservative estimate for customer support), we eliminate $\sim 30\text{--}40\%$ of all vLLM inference requests, directly improving throughput and reducing median latency.

3.2 Optimisation 2 — Chunked Prefill for Low TTFT at High Concurrency

At concurrency 128, multiple long prefill requests may arrive at the same time. By default, vLLM processes each prefill as a single large chunk, which blocks decode steps for all other active sessions until it finishes. This causes other sessions to wait longer for their next token, increasing system-level P95 latency.

We plan to enable chunked prefill (vLLM flag: `-enable-chunked-prefill`), which splits each prefill into smaller chunks and interleaves them with decode steps from other active sessions. The main benefit is for concurrent sessions, which experience less blocking. We note that this may slightly increase TTFT for the request being prefilled, so we expect the primary gain to show up in P95 latency rather than median TTFT. We will tune the chunk size $C \in \{256, 512, 1024\}$ using the training split.

We also plan to enable Automatic Prefix Caching (APC) via `enable_prefix_caching=True`, which reuses KV-cache blocks for shared inputs like the system prompt across sessions.

Tunable parameters:

- Chunk size C : controls the trade-off between decode blocking and GPU utilisation.
- `max_num_seqs`: controls the maximum batch size; we will test values from 64 to 256.
- `gpu_memory_utilization`: we will test values between 0.90 and 0.95, while monitoring for out-of-memory errors under high load.

Expected impact: We expect this optimisation to reduce system-level P95 latency by reducing decode starvation across concurrent sessions. The actual improvement will be measured through experiments on the training split.

3.3 Optimisation 3 — Speculative Decoding with a Draft Model

Customer support replies are usually short (around 20–80 tokens) and often follow repetitive patterns such as “Thank you for reaching out” or “I apologise for the inconvenience.” Because of this predictable structure, speculative decoding can be an effective way to improve generation speed. In speculative decoding, a smaller draft model first proposes a sequence of candidate tokens, which are then verified by the main model in a single forward pass. Tokens that match the target model’s distribution are accepted, while the rest are discarded and resampled.

Implementation plan:

- Use the speculative decoding feature available in vLLM through the `-speculative-model` flag. A lightweight draft model will be used, such as a small distilled variant of the target model or a compact model like Llama-68M.
- Start with a draft length of $k = 4$ tokens and evaluate nearby values ($k \in \{3, 4, 5\}$) based on the observed acceptance rate on the training set.
- For the first message in each session, enable n-gram based speculative decoding as a fallback. vLLM can build an n -gram table from the training data to propose likely tokens for common phrases without relying on a separate draft model.

Quality guarantee: Speculative decoding preserves the exact output distribution of the target model. Tokens that are rejected during verification are simply resampled, meaning the final output quality and perplexity remain unchanged.

Expected impact: Since customer support responses are short and formulaic, speculative decoding can significantly improve decoding throughput. In practice, we expect roughly a $1.5\times$ to $2.5\times$ speedup. The gains are typically larger when the sampling temperature is low, which aligns with the usual settings for customer support systems.

3.4 Optimisation 4 — Session-Aware Continuous Batching Scheduler

The default vLLM scheduler is session-oblivious: it treats each incoming request as independent and schedules based on arrival time and available KV-cache space. For a multi-turn chatbot at concurrency 128, this causes two pathological behaviours:

- **Session interruption:** A session’s mid-generation decode is preempted because a new session with a long prefill arrives, evicting the active session’s KV-cache and requiring recomputation on the next turn.
- **Queue starvation:** A backlog of new prefill requests delays decode completion for all active sessions, inflating per-session end-to-end latency.

We will implement a **session-aware scheduler** that:

- Assigns each active session a *priority score* based on its current turn index (later turns in a conversation are higher priority, as the user is waiting for a reply) and elapsed waiting time (aging).
- Reserves a configurable fraction (e.g. 30%) of each scheduling slot for *decode continuations* of in-flight sessions, preventing decode starvation.
- Implements a *session affinity* mechanism: when a session resumes after an idle period, the scheduler checks whether the session’s KV-cache is still resident before accepting the request. If not, it schedules the prefill recomputation during a low-traffic period (prefetch-on-idle).

Expected impact: Reduction in P95 latency by preventing worst-case session preemption; more predictable TTFT across all concurrency levels.

3.5 Optimisation 5 — Session-TTL KV Cache Eviction Policy

At 128-way concurrency, KV-cache memory is a key bottleneck. The default LRU policy evicts the least recently used block without considering session state. This means it may evict the KV-cache of a session that is simply waiting for the user’s next message, forcing a full recomputation when the user replies.

We plan to modify vLLM’s eviction behaviour to make it aware of session state, using the session-state tracking already introduced in Optimisation 4. The idea is to prioritise evicting sessions that are fully complete or have been idle beyond a configurable TTL, rather than blindly evicting based on recency. Since vLLM’s eviction logic is tied to its PagedAttention scheduler, this will require changes to vLLM’s core scheduling code.

We also plan to explore offloading evicted KV-caches to CPU DRAM instead of discarding them, so that resumed sessions can reload their context rather than recompute from scratch. This trades some resume latency for better memory efficiency, and we will evaluate whether the trade-off is worthwhile through experiments.

Expected impact: We expect fewer unnecessary KV-cache evictions for sessions that are still in use, leading to fewer recomputations and more stable P95 latency under peak concurrency. The actual improvement will be measured on the training split.

4. Implementation Plan

The project spans 5 weeks, with clear ownership assignments across our four-member team. Each member’s primary skills map directly to a subset of the proposed optimisations.

Timeline	Task	Member(s)
Week 8	Baseline profiling & bottleneck analysis	All Members
Week 9	Exact-match + semantic cache (Opt. 1 Tier 1 & 2), Chunked prefill + APC tuning (Opt. 2), Speculative decoding integration (Opt. 3), Session-aware scheduler (Opt. 4), Session-TTL eviction + CPU offload (Opt. 5)	All Members: Each team member will contribute to all project components while working concurrently.
Week 10	Model Performance Benchmarking	All Members
Week 11	Integration & system-level tuning	All Members
Week 12	Vast AI Deployment & Dockerisation & leaderboard submission Final report & video demo	All Members

5. Evaluation Strategy

We evaluate our engine against the official metrics defined for Track B. The results will be measured on the training split locally and validated against the leaderboard using the validation split.

6. Related Work & Positioning

Our approach builds on and extends a body of recent work in LLM serving systems.

- **vLLM / PagedAttention [1]** is our baseline engine. We extend its scheduling and eviction policies without modifying the core attention kernel.
- **Automatic Prefix Caching (APC) [2]** is a built-in vLLM feature we enable as part of Optimisation 2. Our semantic cache (Opt. 1) extends this to fuzzy prefix matching, capturing paraphrase-level repetition that exact hashing misses.
- **Preble [3]** is a distributed LLM serving system targeting long and shared prompts. Its insight that shared KV-cache prefixes should be treated as first-class objects motivates our semantic caching design.
- **InferCept [4]** minimises resource waste during tool-call interruptions in augmented LLM workflows. Our session-aware scheduler addresses the analogous problem of session preemption in the multi-turn chat setting.
- **Speculative Decoding [5]** is the foundational technique underlying Optimisation 3. vLLM’s native implementation makes this straightforward to integrate.
- **Continuum [6]** proposes KV-cache TTL-based management for multi-turn LLM agent scheduling, directly motivating our session-TTL eviction policy in Optimisation 5.

7. Expected Outcomes

We expect our optimisations will improve upon the unmodified vLLM baseline across all official metrics:

- **Avg and P50 latency:** Expected to improve through the semantic cache reducing the number of inference calls.
- **P95 latency:** Expected to improve through chunked prefill and the session-aware scheduler

reducing decode starvation.

- **Throughput:** Expected to increase through the combined effect of caching and speculative decoding.
- **Perplexity:** No degradation expected, as all optimisations are designed to preserve the output distribution of the target model.

The actual improvements will be measured and reported in the final submission. The final deliverable will be a containerised Docker image pushed to GHCR with an immutable SHA-256 digest, submitted alongside a full PDF report, source code archive, and demo video via Canvas before the project deadline.

References

- [1] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *SOSP*, 2023.
- [2] vLLM Project, “Automatic Prefix Caching,” https://docs.vllm.ai/en/stable/design/prefix_caching/, 2024.
- [3] WukLab, “Preble: Efficient Distributed LLM Serving System Targeting Long and Shared Prompts,” *MLSys*, 2025.
- [4] WukLab, “InferCept: Efficient Interceptable Inference Serving for Large Language Models,” in *ICML*, 2024.
- [5] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” in *ICML*, 2023.
- [6] “Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live,” 2025.
- [7] G. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” in *OSDI*, 2022.